PROJECT REPORT

DESIGN AND IMPLEMENTATION OF A CUSTOM 4-BIT CPU IN LOGISIM

Integrating a 4-Bit ALU, Local Register File, Data RAM and Custom Instruction Set Architecture

Course: Digital Logic & Computer Organization Laboratory

Submitted By:

Course – Computer Organization and Architecture

Abstract

This project presents the design of a custom 4-bit central processing unit (CPU) using Logisim Evolution. The design integrates a 4-bit arithmetic logic unit (ALU), a local register file, a data RAM, an instruction ROM, a control unit, a program counter and an instruction register. A custom 12-bit instruction set architecture (ISA) defines data movement, arithmetic, logical, shift, division and control-flow operations. The design is intended to demonstrate the operation of a small programmable processor through instruction fetch, decode and execution. The circuit and ISA spreadsheet supplied with the project form the basis of this report. Functional simulation results should be added after the complete verification program has been run and recorded.

1. Introduction

A central processing unit executes instructions by fetching them from program memory, decoding their operation and controlling the datapath to perform the required task. This project develops a compact 4-bit CPU in Logisim Evolution. Although the data width is four bits, the processor includes a 12-bit instruction word so that the instruction can carry an opcode, register identifiers and an address or immediate field.

The design is divided into modular components: ALU, register file, control unit and top-level CPU. This modular structure makes the datapath and control logic easier to inspect and test.

2. Project Objectives

- Design a 4-bit ALU supporting arithmetic and bitwise operations.
- Create four 4-bit general-purpose registers and an extended HI register.
- Integrate data memory and instruction memory.
- Define a 12-bit custom ISA and its instruction formats.
- Implement instruction sequencing using a program counter and instruction register.
- Demonstrate instruction execution and verify the processor using a self-contained test program.

3. System Architecture

The supplied Logisim project contains four named circuits: ALU, REGFILE, CONTROL and CPU. The CPU is the top-level circuit and instantiates the ALU, register file and control unit. The top-level design also contains a clock, reset input, program counter register, instruction register, data RAM and instruction ROM.

Block	Role / observed configuration
ALU	4-bit datapath; accepts operands A and B and a 4-bit operation select. Exposes result Y, HI and status outputs C, V, N and Z.
Register file	Four 4-bit general-purpose registers R0–R3 and a 4-bit HI register; synchronous write controls and register selection.
Control unit	Instruction decoder/control logic; generates register write, RAM write, HI write, flag write, PC load, halt and HI-select controls.
Program counter	6-bit register, addressing up to 64 instruction words.
Instruction register	12-bit register holding the current instruction.
Instruction ROM	64 words × 12 bits, as indicated by the 6-bit address and 12-bit data configuration.

Data RAM	16 words × 4 bits, using a 4-bit address and 4-bit data configuration.

4. ALU Design

The ALU receives two 4-bit operands and an operation code. The circuit file contains arithmetic components for addition, subtraction, multiplication and division, along with logic and selection components. The interface exposes a 4-bit result Y, a 4-bit HI output and condition outputs C (carry/borrow or operation-specific carry), V (overflow), N (sign/negative) and Z (zero).

Operation	ISA mnemonic	Intended behaviour
Addition	ADD	Rd ← Rd + Rs; updates Z, C, N and V.
Subtraction	SUB	Rd ← Rd − Rs; updates Z, borrow/carry, N and V.
Multiplication	MUL	8-bit product split between HI (upper nibble) and Rd (lower nibble).
Bitwise AND	AND	Rd ← Rd AND Rs; updates Z and N.
Bitwise OR	OR	Rd ← Rd OR Rs; updates Z and N.
Bitwise XOR	XOR	Rd ← Rd XOR Rs; updates Z and N.
Shift left	SHL	Shifts Rd left by one bit; old bit 3 is carried out.
Shift right	SHR	Shifts Rd right by one bit; old bit 0 is carried out.
Division	DIV	Rd receives quotient and HI receives remainder; divide-by-zero is represented through V.

The exact flag equations and timing for each ALU operation should be confirmed against simulation waveforms before final submission. In particular, multiplication and division require checking that both result nibbles are captured correctly.

5. Register Organization

Register	Width	Purpose
R0, R1, R2, R3	4 bits each	General-purpose local registers for operands and intermediate values.
HI	4 bits	Stores the upper product nibble or division remainder.
PC	6 bits	Selects the next instruction in instruction ROM.
IR	12 bits	Holds the instruction currently being decoded/executed.
FLAGS	4 bits	Bit 0 = Z, bit 1 = C, bit 2 = N, bit 3 = V, according to the ISA spreadsheet.

6. Memory Organization

The design uses separate instruction and data memories, corresponding to a Harvard-style organization. The instruction ROM is configured for 64 locations, each 12 bits wide. The data RAM is configured for 16 locations, each 4 bits wide. This separation allows the instruction word width to be 12 bits while data values remain 4 bits.

The 4-bit data address supports addresses 0 through 15. The 6-bit program counter supports instruction addresses 0 through 63.

7. Instruction Set Architecture

The supplied ISA spreadsheet defines a 12-bit instruction word. The fields are [11:8] opcode (4 bits), [7:6] destination/first register Rd (2 bits), [5:4] source register Rs (2 bits), and [3:0] address or immediate (4 bits). JMP and JZ use bits [5:0] as a 6-bit target address. For MOV Rd,HI, bit 0 of the low field selects HI.

Opcode	Mnemonic	Syntax	RTL behaviour
0000	LOAD	LOAD Rd,[addr]	Rd ← RAM[addr]
0001	STORE	STORE Rd,[addr]	RAM[addr] ← Rd
0010	LDI	LDI Rd,imm4	Rd ← imm4
0011	ADD	ADD Rd,Rs	Rd ← Rd + Rs
0100	SUB	SUB Rd,Rs	Rd ← Rd − Rs
0101	MUL	MUL Rd,Rs	HI:Rd ← Rd × Rs
0110	AND	AND Rd,Rs	Rd ← Rd AND Rs
0111	OR	OR Rd,Rs	Rd ← Rd OR Rs
1000	XOR	XOR Rd,Rs	Rd ← Rd XOR Rs
1001	SHL	SHL Rd	Rd ← Rd << 1
1010	SHR	SHR Rd	Rd ← Rd >> 1
1011	DIV	DIV Rd,Rs	Rd ← quotient; HI ← remainder
1100	JMP	JMP target	PC ← target
1101	JZ	JZ target	If Z = 1, PC ← target
1110	MOV	MOV Rd,Rs / MOV Rd,HI	Copies a register or HI to Rd
1111	HALT	HALT	Stops instruction execution

8. Instruction Encoding Examples

Instruction	12-bit word (hex)	Meaning
ADD R0,R1	310	Opcode 3, Rd=0, Rs=1, low field=0.
LOAD R1,[1]	041	Load RAM address 1 into R1.
STORE R0,[5]	105	Store R0 at RAM address 5.
LDI R2,12	28C	Load immediate value C (hex) into R2.
MOV R3,HI	EC1	Copy HI into R3 using the HI-select bit.
JZ target 40	D28	Jump to 6-bit address 40 if Z is set.

9. Instruction Execution

The ISA notes specify a two-clock instruction sequence: one clock for fetch and one clock for execute. During fetch, the instruction at the PC-selected ROM address is read and latched into the instruction register. During execute, the opcode and operand fields are decoded, and the control unit selects the required register, ALU, memory or program-counter action. A normal instruction advances the PC; JMP and JZ can load a target address, while HALT stops further execution.

This two-phase description is taken from the supplied ISA workbook. The exact edge-by-edge implementation should be confirmed from the circuit simulation and timing traces.

10. Control Unit

The CONTROL subcircuit receives the opcode and relevant control inputs, including the zero flag and timing/phase signals. Its outputs include RAMWE, WBSEL, HIWE, FLGWE, RWE, PCLOAD, HALT and HISEL. These signals coordinate register writes, memory writes, flag updates, branch control, halt behaviour and selection of the HI register as a data source.

11. Verification Plan and Test Cases

The assignment requires a self-contained test program that loads operands from data RAM, executes all six baseline ALU operations, demonstrates reuse of local registers in multi-step calculations, executes custom enhancement instructions and writes verified results back to RAM. The following test matrix is recommended. Enter the actual observed values from Logisim after running the program; no simulation results are asserted in this report.

Test	Stimulus / instruction	Expected check
LOAD / STORE	Load a nibble from RAM, then store it at another address.	Destination register and RAM location contain the expected 4-bit value.
LDI / MOV	Load an immediate and copy between registers; also copy HI to a register.	Correct register contents are observed.
ADD / SUB	Test ordinary results and carry/borrow/overflow boundaries.	Result nibble and flags match the defined arithmetic.
MUL	Multiply values whose product fits in 4 bits and values requiring HI.	HI:Rd equals the full 8-bit product.
AND / OR / XOR	Use different bit patterns in two registers.	Bitwise result and Z/N flags are correct.
SHL / SHR	Shift patterns with bit 3 or bit 0 set.	Result and shifted-out carry are correct.
DIV	Test divisible, non-divisible and zero-divisor cases.	Quotient in Rd, remainder in HI; divide-by-zero handled.
JMP / JZ / HALT	Test unconditional jump, taken/not-taken conditional jump and halt.	PC target and halt behaviour are correct.

12. Current Implementation Evidence and Limitations

The supplied Logisim file contains four circuits named ALU, REGFILE, CONTROL and CPU. The CPU top-level contains a 6-bit register, a 12-bit register, 4-bit registers, a 16×4 data RAM and a 64×12 instruction ROM. These structural observations support the architectural description above.

The supplied files alone do not establish that every instruction executes correctly. A successful Logisim simulation, complete test program, and captured output/probe values are still needed. Therefore, this report intentionally does not claim that all test cases have passed. The final submission should include screenshots of the running CPU, PC/IR values, register contents, flags, RAM writes and the test program results.

A point to verify during testing is whether the control-unit timing and datapath implement the stated two-clock fetch/execute sequence consistently. Also verify that the custom instructions SHL, SHR and DIV are fully decoded and that the flags match the ISA table.

13. Conclusion

The project defines and integrates the major components of a compact 4-bit processor: ALU, local registers, control unit, instruction memory, data memory, PC and IR. The 12-bit ISA provides instructions for memory access, immediate loading, arithmetic, logic, shifts, division, data movement and control flow. The modular Logisim architecture provides a basis for studying instruction execution and datapath/control integration. Completion of the required verification program and recording of simulation results are necessary to demonstrate functional correctness.

14. Future Enhancements

- Add a comprehensive automated test program covering every opcode and boundary case.
- Expose probes for PC, IR, registers, HI, flags, ALU buses and RAM interface on the top-level canvas.
- Add additional branch instructions such as jump-if-carry if required.
- Document precise flag semantics for subtraction, multiplication, division and shifts.
- Extend the data memory or instruction set if larger programs or operands are required.